

AGENTE CONVERSACIONAL PARA ADMISIÓN Y NIVELACIÓN DE LA UNIVERSIDAD DE GUAYAQUIL

Grupo 10

Alvarez Echeverria Matthew Alejandro

Angel Eduardo Cuenca Ortega

Procesamiento De Lenguaje Natural

**UNIVERSIDAD DE GUAYAQUIL
FACULTAD DE CIENCIAS MATEMÁTICAS Y FÍSICAS**

10 Julio 2026

1. Introducción

Los procesos de admisión y nivelación de la Universidad de Guayaquil (UG) generan un alto volumen de consultas repetitivas por parte de los aspirantes: requisitos de inscripción, fechas de registro y matrícula, creación de cuentas, aprobación de asignaturas, cupos y aceptación de carreras. Esta información se encuentra dispersa en sitios oficiales, guías y publicaciones en redes, lo que dificulta su consulta ágil.

El presente proyecto propone aplicar técnicas clásicas de Procesamiento de Lenguaje Natural (PLN) para construir un agente conversacional (chatbot) que responda, en español, preguntas frecuentes sobre admisión y nivelación de la UG, a partir de información de fuentes oficiales. El sistema se basa en la detección de intenciones mediante TF-IDF y similitud coseno, con recuperación de respuestas predefinidas, e incorpora una segunda iteración orientada a reducir los errores de clasificación en consultas ambiguas y fuera de dominio.

2. Diseño de Intenciones y Entidades

El sistema define 25 intenciones principales (466 utterances de entrenamiento y 30 respuestas) almacenadas en `data/intents.json`. Cada intención contiene un conjunto de patrones (frases de usuario) y una o varias respuestas predefinidas. Las intenciones cubren:

- Proceso de admisión: registro nacional, inscripción/postulación, evaluación, asignación y aceptación de cupos.
- Proceso de nivelación: requisitos, matrícula, evaluación, plataformas virtuales.
- Información general: oferta académica, costos/aranceles, gratuidad, pérdida de gratuidad, descuentos para grupos vulnerables, reembolso de aranceles, tipos de matrícula, transferencia, requisitos para extranjeros y políticas de cupos.
- Interacción: saludo, despedida, contacto.
- Fallback: intención real entrenada con patrones negativos (fuera de dominio), por ejemplo "cuánto cuesta un taxi para la UG", "noticias", "recetas", para que el clasificador aprenda qué consultas NO pertenecen al dominio del bot.

Las entidades se extraen mediante expresiones regulares y búsqueda por diccionario en `entity_extractor.py`: fechas, cédulas (10 dígitos), montos en dólares, porcentajes, nombres de 57 carreras de la UG y términos de procesos específicos como "curso de nivelación", "cupo aceptado", "matrícula ordinaria", entre otros. Las entidades detectadas se utilizan para enriquecer la respuesta final del agente (por ejemplo, confirmando la carrera mencionada o recordando una fecha).

3. Arquitectura del Agente

La arquitectura del agente sigue un pipeline modular con los siguientes componentes:

- Preprocesamiento (`preprocess.py`): conversión a minúsculas, normalización de acentos, eliminación de puntuación, tokenización, eliminación de stopwords (NLTK) y stemming (Snowball).
- Vectorización (`vectorizer.py`): representación TF-IDF con unigramas y bigramas usando `TfidfVectorizer` de scikit-learn. Se genera una matriz dispersa de las frases de entrenamiento.
- Clasificación (`classifier.py`): cálculo de similitud coseno entre la consulta y todas las frases de entrenamiento, aplicando max-pooling por intención (se toma el mejor puntaje de cada intención, no de cada frase suelta). Se exige superar el umbral de confianza de 0.40 y un margen mínimo de 0.05 entre la mejor y la segunda intención; caso contrario, se retorna el fallback.
- Extracción de entidades (`entity_extractor.py`): identificación de fechas, cédulas, montos, porcentajes, carreras y términos de procesos mediante regex y búsqueda por diccionario, con enriquecimiento de la respuesta.
- Agente central (`agent.py`): coordina el pipeline: preprocesamiento, clasificación, extracción de

entidades y respuesta. Incluye además una consola ejecutable (python src/agent.py) para interactuar desde la terminal.

- Interfaz (app.py): interfaz web con Gradio con identidad institucional: logo, paleta apagada en azul, negro y gris. Incluye un panel izquierdo con preguntas frecuentes de acceso rápido que cambian aleatoriamente en cada carga de la página, botones Enviar y Limpiar conversación, persistencia del historial por navegador y valoración de respuestas (me gusta / no me gusta) registrada en data/feedback.csv. No expone al usuario la intención detectada ni el nivel de confianza.

- Evaluación (evaluate.py): script con 83 consultas de prueba que calcula accuracy, F1-macro, reporte de clasificación y matriz de confusión.

4. Justificación de Decisiones de Diseño

a) TF-IDF vs bolsa de palabras con conteos simples.

TF-IDF pondera la frecuencia de un término por su importancia inversa en el corpus. El factor IDF reduce el peso de palabras comunes (como "universidad" o "estudiante") y aumenta el peso de términos discriminativos (como "nivelación", "cupos", "aranceles"). Esto permite diferenciar mejor entre intenciones que comparten vocabulario común, como "matrícula de nivelación" y "tipos de matrícula".

b) Similitud coseno vs distancia euclidiana.

La similitud coseno mide el ángulo entre dos vectores, no su magnitud, por lo que es invariable a la longitud del documento. La distancia euclidiana depende de la longitud: una frase larga estaría más lejos de una corta aunque tengan el mismo significado. En nuestro caso, consultas de distinta longitud deben compararse independientemente de la cantidad de palabras.

c) Preprocesamiento antes de vectorizar.

El preprocesamiento (stemming y eliminación de stopwords) reduce el vocabulario y agrupa formas flexionadas. Por ejemplo, "inscripción", "inscribirme" e "inscribiendo" se reducen a la raíz "inscrib", mejorando la coincidencia. Las stopwords como "el", "la", "de" no aportan significado y se eliminan para reducir ruido en la representación TF-IDF.

d) Umbral de confianza, margen mínimo y fallback entrenado.

En la primera versión se usó un umbral global de 0.35 y se seleccionó la intención con la mayor similitud entre todas las frases. Esto producía falsos positivos en consultas fuera de dominio: por ejemplo, "cuánto cuesta un taxi para la UG" se clasificaba como matrícula de nivelación y "noticias de la universidad" como transferencia. Para corregirlo se adoptaron tres medidas: (1) el umbral se elevó a 0.40, (2) se añadió un margen mínimo de 0.05 entre la mejor y la segunda intención, de modo que ante una consulta ambigua el agente prefiere el fallback antes que arriesgar una respuesta incorrecta, y (3) el fallback se convirtió en una intención real con ejemplos negativos (taxi, pasaje, noticias, comida, clima), permitiendo que el clasificador aprenda el vocabulario que no pertenece al dominio.

e) Max-pooling por intención.

Comparar la consulta contra cada frase suelta favorece los patrones largos o duplicados. Al agrupar los puntajes por intención y tomar el máximo de cada grupo, se neutraliza ese sesgo y se mejora la separación entre intenciones que comparten vocabulario (por ejemplo, asignación de cupos vs políticas de cupos vs aceptación de cupo), que en el análisis inicial presentaban similitudes cruzadas de hasta 1.0 por patrones duplicados.

f) JSON separado del código.

Almacenar intenciones, utterances y respuestas en JSON permite: modificar patrones y respuestas sin tocar el código fuente, facilitar la adición de nuevas intenciones, mantener la separación de responsabilidades (datos vs lógica) y permitir que múltiples componentes (vectorizer, agent) accedan

a los mismos datos.

g) Manejo de entradas problemáticas con try-except.

El agente maneja entradas problemáticas en múltiples niveles: consulta vacía o en blanco, texto sin tokens después del preprocesamiento, y similitud por debajo del umbral o sin margen suficiente. Estas validaciones se implementan en agent.py antes de cada paso del pipeline, evitando que el agente se detenga ante entradas inválidas.

5. Resultados de Evaluación

El agente se evaluó con un conjunto ampliado de 83 consultas que incluye: (1) reformulaciones no vistas en entrenamiento, (2) pares confusos entre intenciones similares (matrícula de nivelación vs costos/aranceles, gratuidad vs pérdida de gratuidad, asignación vs aceptación de cupos), (3) consultas fuera de dominio simples y adversariales, y (4) errores tipográficos. Los resultados obtenidos fueron:

Métrica	Primera versión (28 consultas)	Versión actual (83 consultas)
Accuracy	92.86%	100.00%
F1-Macro	95.83%	100.00%
Correctas	26/28	83/83
Falsos positivos OOD	2	0

En la primera versión, los 2 errores correspondían a consultas OOD con vocabulario del dominio ("universidad", "cuesta") que no eran sobre admisión o nivelación. El rediseño (fallback entrenado con patrones negativos + margen mínimo) eliminó esos errores: en las 83 consultas no se registró ningún falso positivo fuera de dominio. Además, un reentrenamiento completo sobre las 466 utterances clasificó la totalidad de los patrones correctamente, y las baterías de validación descritas a continuación se ejecutaron sin fallos.

5.1 Baterías de Validación Adicionales

Para fortalecer la evidencia de robustez, el sistema se sometió a baterías complementarias además de la suite de 83 consultas:

Prueba	Resultado
Reentrenamiento completo (auto-train, 466 utterances)	466/466 correctos
Conjunto adversarial 1 (24 paráfrasis)	24/24
Conjunto adversarial 2 (40 paráfrasis nuevas)	40/40
Extracción de entidades	6/6
Casos límite (vacías, cortas, largas, sin tokens)	11/11
Fuera de dominio -> fallback (OOD)	11/11
Pruebas de seguridad (XSS, prompts, PII, fuzzing, spoofing)	0 fallos
Rendimiento promedio por consulta	0.84 ms

La batería de seguridad ejecutó más de 300 pruebas automatizadas que cubren: inyección de scripts y HTML (XSS), intentos de inyección de prompt, exposición de datos personales (cédulas), entradas maliciosas (bytes nulos, texto bidireccional, cadenas de 100 000 caracteres) y consultas de suplantación o fuera de dominio. El sistema no registró fallos: no existe ningún vector de ejecución de código, no se exponen secretos en el repositorio ni en el historial, y el despliegue de la interfaz no habilita la opción de compartir el enlace públicamente.

Limitaciones observadas:

- Dependencia de patrones: el sistema solo reconoce intenciones para las cuales tiene patrones de entrenamiento; paráfrasis muy alejadas pueden caer en fallback.
- Sin contexto: no mantiene estado entre turnos conversacionales.
- Sin comprensión semántica: TF-IDF compara términos, no significado; sinónimos no entrenados pueden fallar.
- OOD con vocabulario del dominio: consultas como "cuánto cuesta un taxi para la UG" siguen siendo un reto, aunque el margen mínimo reduce notablemente estos casos.

6. Conclusiones

El agente conversacional demuestra que técnicas clásicas de PLN como TF-IDF y similitud coseno son suficientes para construir un chatbot funcional en un dominio específico. El sistema alcanza un accuracy del 100% en un conjunto ampliado de 83 consultas, con 25 intenciones y 466 patrones de entrenamiento, cubriendo los principales temas de admisión y nivelación de la UG.

Las mejoras más relevantes de la segunda iteración fueron: entrenar el fallback con ejemplos negativos para rechazar consultas fuera de dominio, aplicar max-pooling por intención para reducir la confusión entre intenciones similares, e incorporar un margen mínimo de confianza. Estas medidas eliminaron los falsos positivos OOD sin sacrificar la cobertura de consultas legítimas.

Las principales limitaciones están relacionadas con la falta de comprensión semántica profunda y la ausencia de memoria conversacional. Sin embargo, para un asistente virtual de FAQ en un dominio restringido, el enfoque demostrado es efectivo y replicable. Los aprendizajes incluyen la importancia del preprocesamiento de texto, la selección adecuada del umbral y del margen de confianza, y la necesidad de un conjunto de prueba variado que incluya consultas adversariales para una evaluación realista.

Referencias

1. Coordinación de Admisión y Nivelación. (s.f.). Portal oficial de admisión Universidad de Guayaquil. <https://admision.ug.edu.ec/admision/>
2. Coordinación de Admisión y Nivelación. (s.f.). Nivelación Universidad de Guayaquil. <https://admision.ug.edu.ec/nivelacion/>
3. Coordinación de Admisión y Nivelación. (s.f.). Oferta académica UG. <https://admision.ug.edu.ec/oferta-ug/>
4. Coordinación de Admisión y Nivelación. (s.f.). Reglamento de admisión de la Universidad de Guayaquil 2023 (Reformado) [PDF]. Universidad de Guayaquil.
5. Coordinación de Admisión y Nivelación. (s.f.). Reglamento de matrículas, aranceles y derechos de la Universidad de Guayaquil 2023 (Reformado) [PDF]. Universidad de Guayaquil.
6. alaU. (s.f.). Todo lo que necesitas saber para ingresar a la Universidad de Guayaquil. Blog alaU. <https://blog.alau.org/todo-lo-que-necesitas-saber-para-ingresar-a-la-universidad-de-guayaquil/>